



Topic 10: Data Link Control

CSE 4405: Data and Telecommunications

Md. Tanvir Hossain Saikat, Jr. Lecturer

Islamic University of Technology (IUT)

Chapter Roadmap

Core questions

- How does a bit stream become separate frames?
- How does the sender avoid overwhelming the receiver?
- How do link protocols recover from damaged or lost frames?
- How do HDLC frames carry data, control, and management information?

Original chapter order

1. Framing
2. Flow and error control
3. Protocol taxonomy
4. Noiseless channels
5. Noisy channels
6. Piggybacking and HDLC

Lecture view: data link control is node-to-node communication. Each link must package, pace, check, and sometimes retransmit frames before the next hop receives useful data.

10.1 Framing

Why framing exists

The data link layer packs raw bits into **frames** so that one unit of information can be distinguished from the next.

- A frame is similar to an envelope in a postal system.
- The envelope separates one message from another.
- In data communication, the frame boundary tells the receiver where one unit starts and ends.

Topics in this section

- Fixed-size framing
- Variable-size framing

Framing Choices

Fixed-size framing

The frame length is known in advance, so the size itself identifies the boundary.

- No separate delimiter is required.
- The receiver counts a fixed number of bits or bytes per frame.

Variable-size framing

Frame length can change, so the protocol needs an explicit boundary marker.

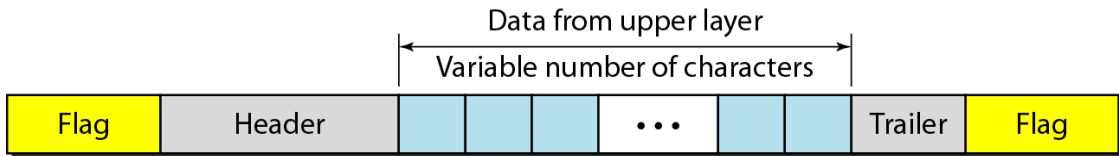
- Character-oriented protocols view data as bytes.
- Bit-oriented protocols view data as a sequence of bits.

The rest of this section explains how variable-size frames avoid confusing data with delimiters.

Character-Oriented Protocols

- Data are carried as 8-bit characters.
- A special 8-bit flag is placed at the beginning and end of each frame.
- The flag value is protocol dependent.
- The receiver uses the flags to find frame boundaries.

Figure: 10.1 A frame in a character-oriented protocol



The flag is not payload; it is a delimiter wrapped around the header, data, and trailer.

Byte Stuffing

The delimiter problem

A flag pattern used for frame boundaries can also appear inside the information field. Without extra handling, the receiver may mistake data for the end of the frame.

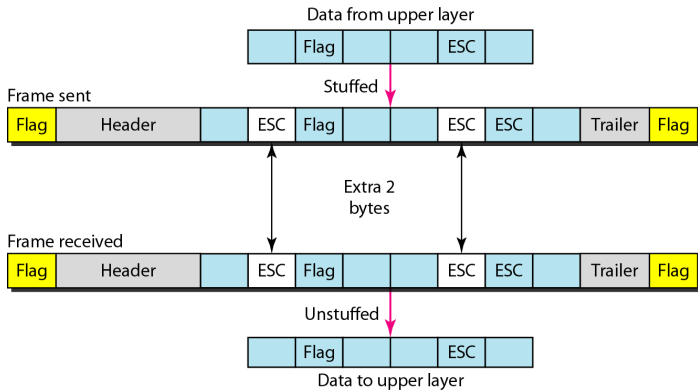
- Character-oriented protocols solve this with **byte stuffing**.
- The sender inserts an escape byte before a data byte that looks like a flag or escape character.
- The receiver removes the inserted escape byte and treats the following byte as data.
- Escape characters that are part of the data must also be escaped.

Byte stuffing: add one extra ESC byte whenever the payload contains a flag byte or an ESC byte.

Byte Stuffing and Unstuffing

- The sender protects special byte patterns before transmission.
- The receiver removes only the inserted ESC bytes.
- The original payload is recovered without losing real flag-like data.

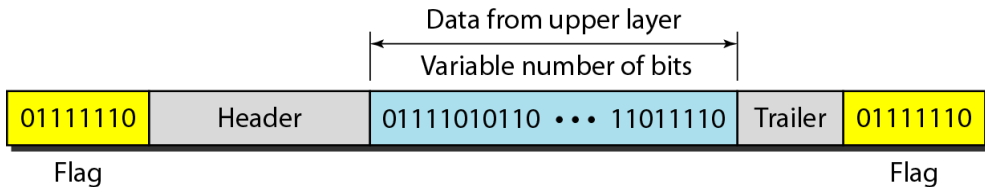
Figure: 10.2 Byte stuffing and unstuffing



Bit-Oriented Protocols

- The data field is treated as a sequence of bits.
- A common delimiter is the 8-bit flag 01111110.
- The same flag marks both the beginning and the end of the frame.
- Upper layers may read the payload as text, graphics, audio, or video.

Figure: 10.3 A frame in a bit-oriented protocol



One flag pattern locates both frame boundaries.

Bit Stuffing

The bit-level version of stuffing

The delimiter 01111110 must not appear accidentally inside the payload. Bit-oriented protocols prevent this by inserting one extra bit.

Sender rule

Whenever five consecutive 1s appear in the data stream, insert one 0.

Receiver rule

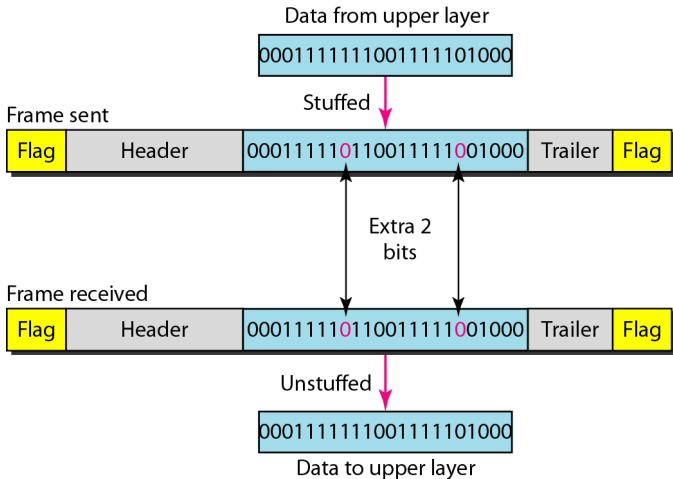
Whenever five consecutive 1s followed by a stuffed 0 are seen, remove that 0.

Result: the flag pattern remains unique as a frame boundary.

Bit Stuffing and Unstuffing

- Stuffing prevents data from imitating the delimiter.
- Unstuffing restores the exact original data bits.
- The receiver can detect true flags and ignore false ones.

Figure: 10.4 Bit stuffing and unstuffing



10.2 Flow and Error Control

Data link control

The most important data-link responsibilities are **flow control** and **error control**. Together, they are called data link control.

Flow control asks

- Is the sender transmitting too fast?
- Can the receiver buffer and process the frames?

Error control asks

- Did a frame arrive damaged?
- Was a frame or acknowledgment lost?

Definition

Flow control is the set of procedures that restrict how much data a sender may transmit before waiting for feedback.

- The receiver has finite processing speed and finite buffer space.
- A fast sender can overwhelm a slower receiver.
- Feedback or window limits regulate the sender's rate.
- The goal is to keep the receiver useful, not merely keep the link busy.

Definition

Error control combines error detection with a recovery method. At the data-link layer, recovery is commonly based on **automatic repeat request** (ARQ).

- Detection checks whether received information is valid.
- ARQ retransmits data when a frame or acknowledgment is damaged or lost.
- Sequence numbers help identify duplicates and missing frames.
- Timers help the sender recover when no response arrives.

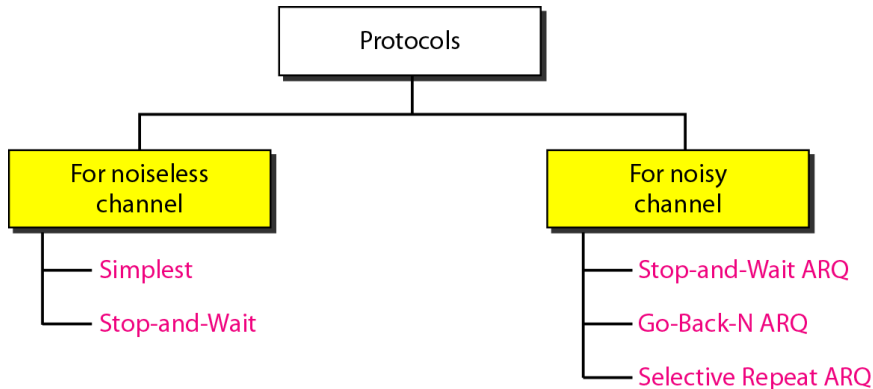
10.3 Protocols

- A data-link protocol combines framing, flow control, and error control.
- The goal is delivery from one node to the next over a single link.
- Real implementations are written in software or hardware logic.
- In the chapter, pseudocode is used so the protocol ideas are language independent.

How to read the protocols

Each later protocol fixes a limitation of an earlier one: first ideal links, then slower receivers, then noisy channels, then more efficient pipelining.

Figure: 10.5 Taxonomy of protocols discussed in this chapter



The tree splits on one question: can the channel damage or lose a frame?

10.4 Noiseless Channels

Temporary assumption

Assume an ideal channel: no frames are lost, duplicated, or corrupted.

Why start here?

- It isolates basic sender/receiver coordination.
- It shows why flow control is needed even when errors do not occur.
- It prepares the structure used later by ARQ protocols.

Protocols in this section

- Simplest Protocol
- Stop-and-Wait Protocol

Simplest Protocol: Assumptions

- The Simplest Protocol has no flow control and no error control.
- Data frames travel in only one direction: sender to receiver.
- The receiver can handle every incoming frame immediately.
- Receiver processing time is assumed negligible.
- Under these assumptions, the receiver is never overwhelmed.

Teaching point

This protocol is intentionally unrealistic; it is a baseline for seeing what later mechanisms add.

Simplest Protocol: Events

- The sender uses services from its physical layer and serves its network layer.
- The sender can transmit only when its network layer has a packet ready.
- The receiver can deliver a packet upward only after a frame arrives.
- There is no acknowledgment, timer, or retransmission.
- Event-driven pseudocode is used to model these actions.

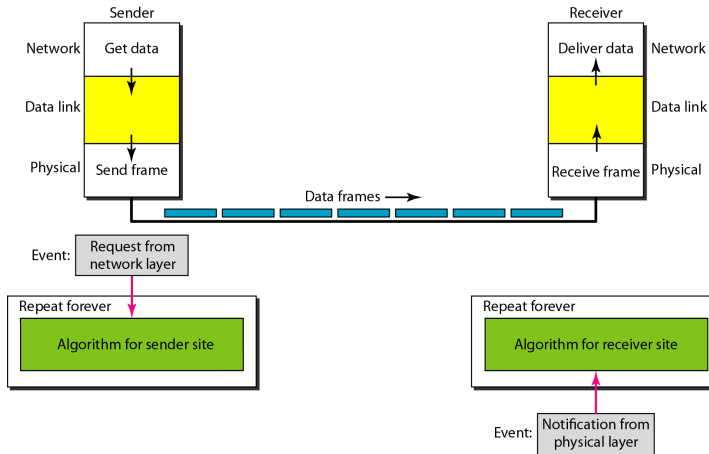
Event view

A protocol mostly sleeps until something happens: a request from the network layer or an arrival from the physical layer.

Simplest Protocol Design

- Network-layer packets move down at the sender.
- Frames cross the physical link.
- Payloads are delivered up at the receiver.
- There is no feedback path.

Figure: 10.6 Design with no flow or error control



Algorithm 10.1: Simplest Protocol Sender

```
while true do
  wait for an event

  if event == request_from_network_layer then
    packet <- get_data()
    frame  <- make_frame(packet)
    send_frame(frame)
  end if
end while
```

The sender accepts data from the network layer and immediately sends a frame. It does not wait for any receiver feedback.

Algorithm 10.2: Simplest Protocol Receiver

```
while true do
  wait for an event

  if event == frame_arrival then
    frame <- receive_frame()
    packet <- extract_data(frame)
    deliver_data(packet)
  end if
end while
```

The receiver accepts every arriving frame and immediately delivers the payload upward.

Example 10.1: Simplest Protocol

- The sender transmits a sequence of frames without considering receiver state.
- Three sender events send three frames.
- Three receiver events accept and deliver those frames.
- The tilted boxes in the flow diagram represent transmission time.
- The box height shows the time gap between the first bit and last bit of a frame.

What the example teaches

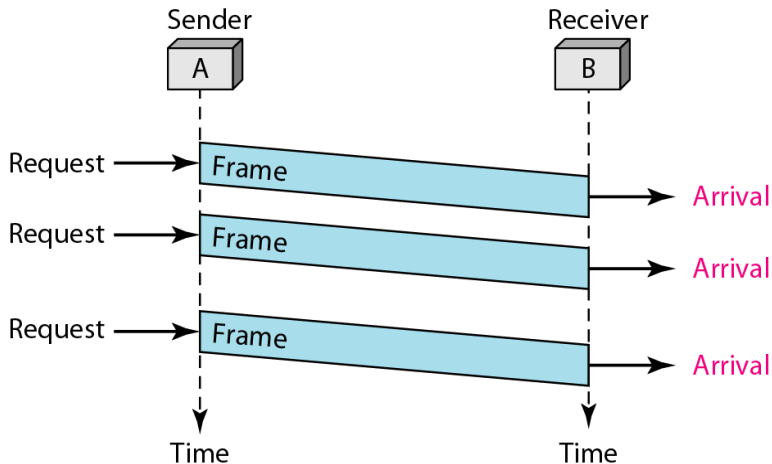
With ideal assumptions, communication is just one-way frame movement. The model breaks as soon as the receiver can be slower than the sender.

Example 10.1 Flow Diagram

Figure: 10.7 Flow diagram for Example 10.1

- Three sender events send three frames.
- Each tilted box is one frame in transit.
- Box height is the first-bit to last-bit time.

The sender never pauses for feedback, so all coordination is one-way.



Stop-and-Wait Protocol

Add flow control

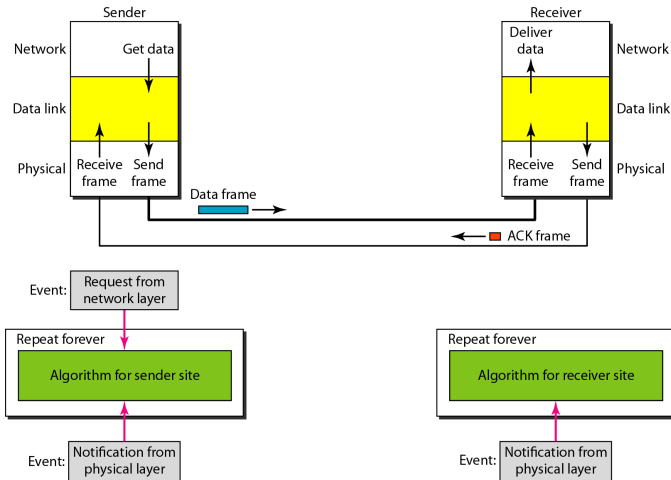
To prevent a receiver from being overwhelmed, the sender transmits one frame and waits for an acknowledgment before sending the next one.

- Data frames still move from sender to receiver.
- Auxiliary ACK frames move in the reverse direction.
- The sender cannot send the next data frame until an ACK arrives.
- This is still a noiseless-channel protocol: ACKs are assumed reliable.

Stop-and-Wait Design

- The sender keeps only one outstanding frame.
- The receiver returns an ACK after accepting the frame.
- The reverse ACK path enables flow control.

Figure: 10.8 Design of Stop-and-Wait Protocol



Algorithm 10.3: Stop-and-Wait Sender

```
while true do
  wait for an event

  if event == request_from_network_layer then
    packet <- get_data()
    frame  <- make_frame(packet)
    send_frame(frame)
    block_new_requests_until_ack()
  end if

  if event == ack_arrival then
    allow_next_request()
  end if
end while
```

The ACK acts as permission to send the next frame.

Algorithm 10.4: Stop-and-Wait Receiver

```
while true do
  wait for an event

  if event == frame_arrival then
    frame <- receive_frame()
    packet <- extract_data(frame)
    deliver_data(packet)
    send_ack()
  end if
end while
```

The receiver confirms every frame it accepts, enabling the sender to continue.

Example 10.2: Stop-and-Wait

- The sender transmits one frame and then waits.
- After the ACK arrives, the sender transmits the next frame.
- Sending two frames creates four sender-side events.
- The receiver has two frame-arrival events.
- The link may be idle while the sender waits for ACKs.

What the example teaches

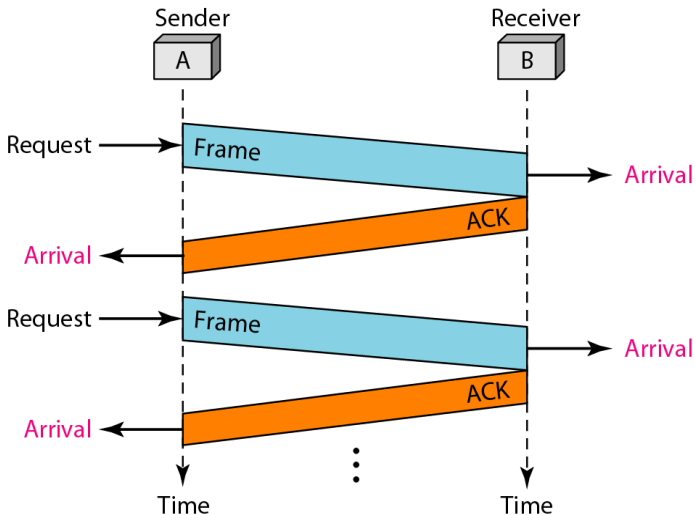
Stop-and-Wait protects the receiver, but it can waste capacity when the round-trip delay is large.

Example 10.2 Flow Diagram

Figure: 10.9 Flow diagram for Example 10.2

- Two frames produce four sender events.
- The receiver has two frame-arrival events.
- The reverse arrows are the ACK frames.

Each data frame is followed by a waiting period for its ACK, during which the link is idle.



10.5 Noisy Channels

Real channels are noisy

The noiseless Stop-and-Wait Protocol gives flow-control intuition, but real links can lose, duplicate, or corrupt frames.

Protocols in this section

- Stop-and-Wait Automatic Repeat Request
- Go-Back-N Automatic Repeat Request
- Selective Repeat Automatic Repeat Request

All three add error control; the main difference is how many frames may be outstanding and how retransmissions are handled.

Stop-and-Wait ARQ

Add error control to Stop-and-Wait

The sender keeps a copy of each transmitted frame and retransmits it if the timer expires before a valid acknowledgment arrives.

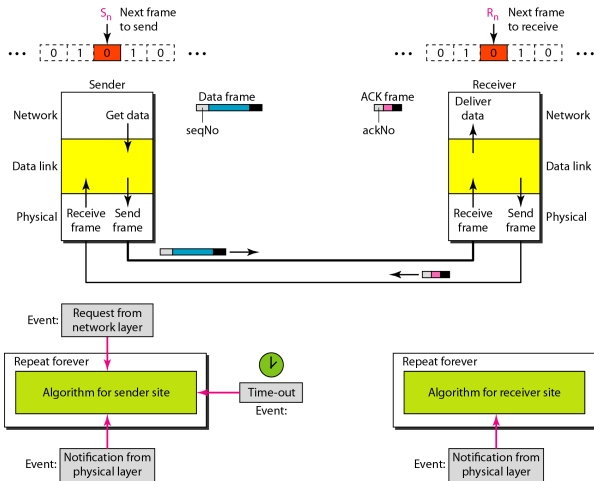
- Frames use sequence numbers based on modulo-2 arithmetic: 0, 1, 0, 1, ...
- The acknowledgment number indicates the next frame expected.
- If frame x is accepted, the receiver sends ACK $(x + 1) \bmod 2$.
- A corrupted or lost frame must be resent.
- A lost ACK can also cause a retransmission, so sequence numbers prevent duplicate delivery.

In Stop-and-Wait ARQ, the sender sends one frame at a time and waits for an acknowledgement before sending the next one. Each frame carries a sequence number, alternating between 0 and 1. After receiving a frame correctly, the receiver sends an ACK indicating the sequence number it expects next. If a frame or ACK is lost, the sender's timer expires and it retransmits the same frame with the same sequence number. The receiver can then detect that it is an old retransmission because it is expecting the other sequence number, so it discards the duplicate instead of delivering the data twice. This way, sequence numbers, ACKs, and timers work together to provide reliable delivery.

Stop-and-Wait ARQ Design

- The sender stores a copy.
- A timer bounds waiting time.
- ACKs carry the next expected sequence number.
- Retransmission handles loss or damage.

Figure: 10.10 Design of Stop-and-Wait ARQ



Algorithm 10.5: Stop-and-Wait ARQ Sender (Send)

```
Sn <- 0                                // sequence number of next frame
can_send <- true

while true do
  wait for an event

  if event == request_from_network_layer and can_send then
    packet <- get_data()
    frame <- make_frame(packet, seq = Sn)
    store_copy(frame)
    send_frame(frame)
    start_timer()
    can_send <- false
  end if

  // the same event loop continues on the next slide
```

When a packet arrives from the network layer, the sender transmits one numbered frame, stores a copy, and starts a timer.

Algorithm 10.5: Stop-and-Wait ARQ Sender (ACK/Timeout)

```
// continued: the same event loop started on the previous slide

if event == ack_arrival and ack_is_valid(next = (Sn + 1) mod 2) then
    stop_timer()
    discard_copy()
    Sn <- (Sn + 1) mod 2
    can_send <- true
end if

if event == timeout then
    resend_stored_frame()
    restart_timer()
end if
end while
```

A valid ACK advances the sender; a timeout retransmits the stored frame.

Algorithm 10.6: Stop-and-Wait ARQ Receiver

```
Rn <- 0                                // sequence number expected

while true do
  wait for a frame arrival

  frame <- receive_frame()

  if frame_not_corrupted(frame) and seq(frame) == Rn then
    packet <- extract_data(frame)
    deliver_data(packet)
    Rn <- (Rn + 1) mod 2
    send_ack(Rn)
  else
    // Damaged frame or duplicate frame.
    // Repeating ACK(Rn) tells the sender what is still expected.
    send_ack(Rn)
  end if
end while
```

this acts as an NAK and is not strictly necessary, it is given to improve efficiency so that the sender doesn't have to wait the entire timeout period to know that the frame needs to be retransmitted

Example 10.3: Stop-and-Wait ARQ

- Frame 0 is sent and acknowledged successfully.
- Frame 1 is lost and retransmitted after timeout.
- The retransmitted frame 1 is acknowledged and its timer stops.
- A later frame 0 is received, but its ACK is lost.
- The sender cannot know whether the frame or ACK was lost, so it retransmits frame 0 after timeout.

What the example teaches

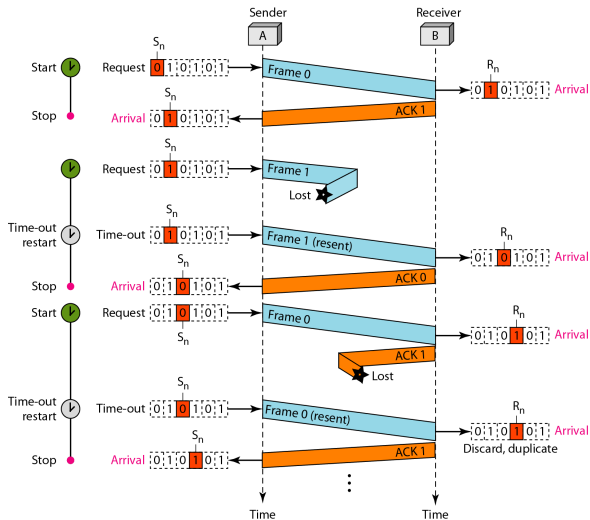
A timeout means “something went wrong,” not necessarily “the data frame was lost.” Sequence numbers make retransmission safe.

Example 10.3 Flow Diagram

Figure: 10.11 Flow diagram for Example 10.3

- Frame 0 is sent and acknowledged normally.
- Frame 1 is lost, so the timer expires and frame 1 is resent.
- The next frame 0 arrives, but its ACK is lost, which triggers a second timeout.

The diagram shows both a lost data frame and a lost ACK, handled by the same timeout-and-retransmit logic.



Bandwidth-Delay Product

Definition

The bandwidth-delay product estimates how many bits can be in transit while the sender waits for feedback.

Bandwidth-delay product = bandwidth $\times$ round-trip delay

$$\text{Utilization} = \frac{\text{frame length}}{\text{bandwidth-delay product}} \times 100\%$$

A large product means Stop-and-Wait can leave most link capacity unused.

Example 10.4: Stop-and-Wait Utilization

Given

- Bandwidth = 1 Mbps
- Round-trip time = 20 ms
- Frame length = 1000 bits

Calculation

$$1,000,000 \text{ bits/s} \times 0.020 \text{ s} = 20,000 \text{ bits}$$

$$\text{Utilization} = \frac{1000}{20,000} \times 100\% = 5\%$$

Conclusion: during one round trip, the link could carry 20,000 bits, but Stop-and-Wait sends only one 1000-bit frame.

Example 10.5: Why Pipelining Helps

Question

- Keep the same bandwidth-delay product: 20,000 bits.
- Allow the sender to transmit up to 15 frames before waiting.
- Each frame is 1000 bits.

Calculation

$$15 \times 1000 = 15,000 \text{ bits} \quad \text{Utilization} = \frac{15,000}{20,000} \times 100\% = 75\%$$

Damaged frames still reduce real utilization, but sending multiple outstanding frames can fill much more of the path.

Pipelining

Definition

Pipelining starts a new transmission task before the previous one has fully completed.

- Stop-and-Wait ARQ has no pipelining.
- Go-Back-N ARQ and Selective Repeat ARQ use pipelining.
- Pipelining improves efficiency when many bits can be in transit.
- The gain is most visible when the bandwidth-delay product is large.

Tradeoff: pipelining improves utilization, but the protocol must now track multiple outstanding frames.

Go-Back-N ARQ: Sequence Numbers

- Go-Back-N ARQ allows several frames to be sent before ACKs arrive.
- The sender keeps copies of outstanding frames.
- Frames are numbered sequentially using m bits.
- Sequence numbers range from 0 to $2^m - 1$, then repeat modulo 2^m .
- Example with $m = 4$: 0, 1, ..., 15, 0, 1, ...

Main idea

The sender may move ahead, but it must be ready to go back and retransmit from the first missing frame.

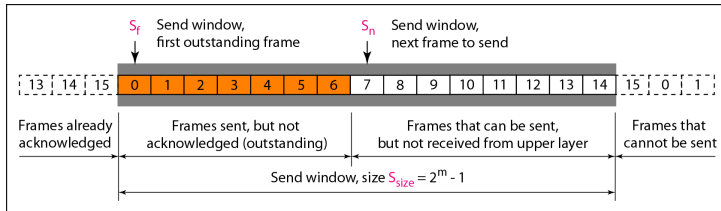
Sliding window

A sliding window identifies the sequence numbers currently relevant to the sender or receiver.

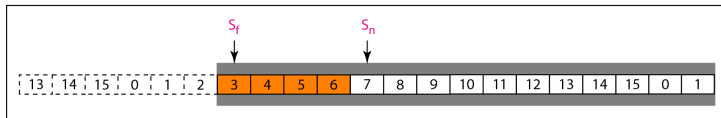
- The send window has size $2^m - 1$.
- S_f marks the first outstanding frame.
- S_n marks the next frame to send.
- S_{size} is the send-window size.
- The window slides forward when valid acknowledgments arrive.

Go-Back-N Send Window

Figure: 10.12 Send window for Go-Back-N ARQ



a. Send window before sliding



b. Send window after sliding

The sender in Go-Back-N ARQ uses a sliding window to send multiple frames without waiting for an ACK after each one. S_f represents the oldest unacknowledged frame, while S_n represents the next frame to send. ACKs are cumulative, so when an ACK arrives, the sender can slide the window forward. If the timer for the oldest outstanding frame expires, the sender retransmits that frame and all later unacknowledged frames.

For example: if ACK6 is received back from the receiver this means that the sender has received everything correctly and in order from frame 0-5 and is now expecting frame 6, so the sender can slide the window forward by pushing S_f to 6, and transmit every frame from bit 6 to S_n-1 .

The sender may transmit any frame whose sequence number falls inside the current send window.

Go-Back-N ARQ: Receive Window

- The receive window size is always 1.
- R_n stores the one sequence number currently expected.
- If the correct frame arrives, the receiver accepts it and slides one slot.
- If a frame arrives out of order, the receiver discards it.
- Discarded out-of-order frames must be retransmitted later.

The receiver in Go-Back-N has a receive window size of only 1, meaning it accepts only the exact frame it is currently expecting. R_n stores the sequence number of that expected frame. If the correct frame arrives, the receiver accepts it, delivers the data, and moves to the next sequence number. If a frame arrives out of order, it is discarded, and the receiver continues waiting for the missing frame.

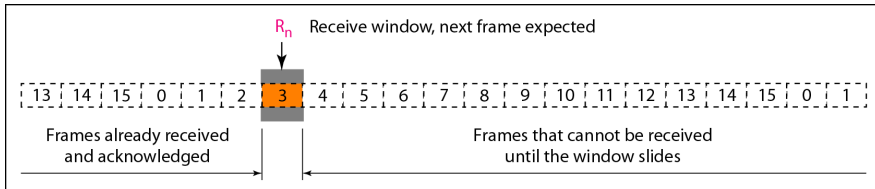
NOTE: The receiver sends an ACK for every frame it receives, not just one ACK for one window. The sender takes into account the latest ACK it receives and discards earlier ACK's to find the latest frame the sender has received correctly

Why it is called Go-Back-N

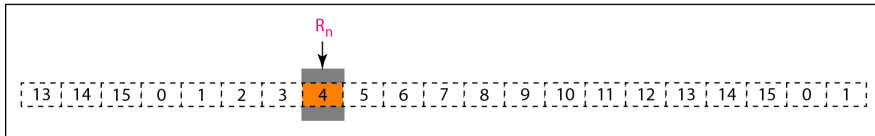
After a loss, the sender retransmits the missing frame and all later outstanding frames.

Go-Back-N Receive Window

Figure: 10.13 Receive window for Go-Back-N ARQ



a. Receive window



b. Window after sliding

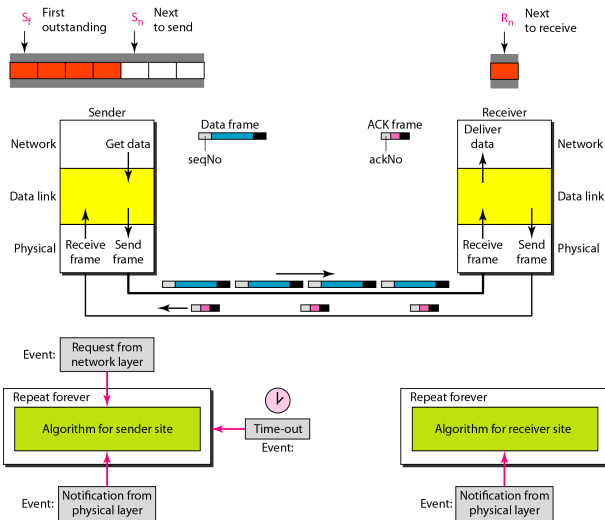
The receiver accepts only the next expected frame; all other arrivals are outside the window and are discarded.

Go-Back-N ARQ Design

Figure: 10.14 Design of Go-Back-N ARQ

- The sender holds a window of stored, unacknowledged frames.
- One timer runs for the oldest outstanding frame.
- The receiver keeps a single expected sequence number R_n .

Multiple outstanding frames require a send window, cumulative ACKs, and retransmission from the first unresolved frame.

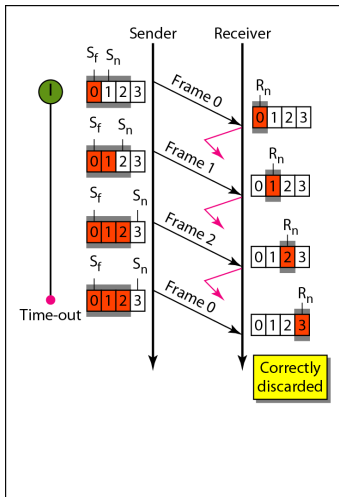


Go-Back-N Window Size

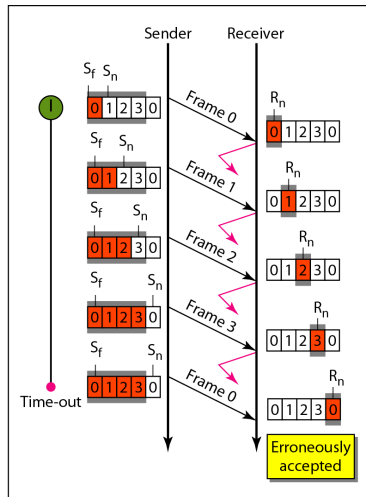
Figure: 10.15 Window size for Go-Back-N ARQ

- Part (a) uses a legal window size of $2^m - 1$.
- Part (b) uses 2^m and the receiver wrongly accepts a duplicate.

The send window must be smaller than the full sequence-number space to avoid ambiguity after numbers wrap around.



a. Window size $< 2^m$



b. Window size $= 2^m$

Go-Back-N Window Rule

Send-window size $< 2^m$ Receive-window size = 1

- m is the number of bits in the sequence-number field.
- Sender window can contain at most $2^m - 1$ frames.
- Receiver window remains one frame because out-of-order frames are discarded.
- Stop-and-Wait ARQ is the special case where the send-window size is 1.

Example 10.6: Go-Back-N with Lost ACK

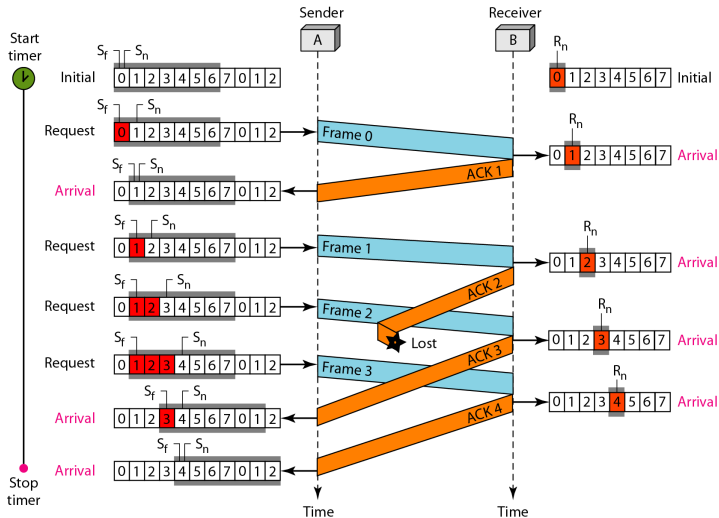
- The forward channel is reliable, but the reverse channel is not.
- No data frames are lost.
- Some ACKs are delayed and one ACK is lost.
- Cumulative acknowledgments help recover from delayed or lost ACKs.
- ACK 3 confirms both frame 2 and frame 3, so losing ACK 2 is not fatal.
- No timeout occurs because all outstanding frames are acknowledged before the timer expires.

Example 10.6 Flow Diagram

Figure: 10.16 Flow diagram for Example 10.6

- No data frame is lost; only ACKs are delayed or lost.
- ACK 3 covers frames 2 and 3 together.
- No timeout fires, so nothing is retransmitted.

Cumulative ACKs allow later acknowledgments to cover earlier ones.



Example 10.7: Go-Back-N with a Lost Frame

- Frames 0, 1, 2, and 3 are sent.
- Frame 1 is lost.
- The receiver discards frames 2 and 3 because they arrive out of order.
- The sender receives no ACK for frames 1, 2, or 3.
- When the timer expires, the sender retransmits all outstanding frames starting from frame 1.
- One timeout event causes the retransmission of frames 1, 2, and 3.

Example 10.7: Event Ordering Detail

- While the sender is responding to one event, new arrivals must wait.
- ACK 2 may arrive while the sender is still retransmitting frame 3.
- ACK 3 may arrive while the sender is processing ACK 2.
- The diagram uses vertical delay marks to show this waiting.
- Before the second timer expires, all outstanding frames have been sent and the timer stops.

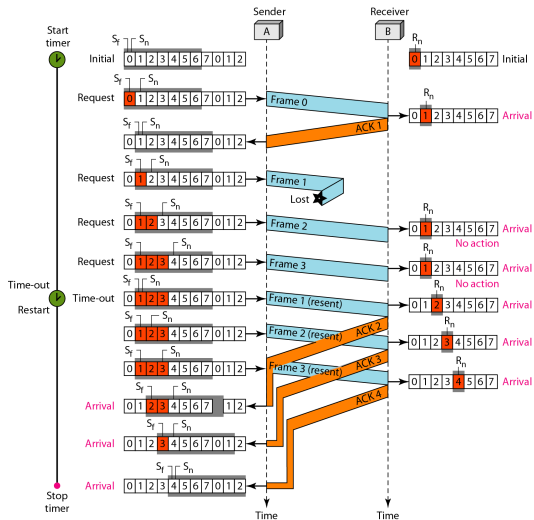
Protocol behavior depends not only on frame order, but also on how events are serialized at each station.

Example 10.7 Flow Diagram

Figure: 10.17 Flow diagram for Example 10.7

- Frames 0 to 3 are sent; frame 1 is lost.
- Frames 2 and 3 arrive out of order and are discarded.
- One timeout resends frames 1, 2, and 3.
- The vertical delay marks show events queued while the sender is busy.

A single missing frame forces Go-Back-N to resend that frame and every later outstanding frame.



Selective Repeat ARQ: Motivation

- Go-Back-N ARQ is inefficient on noisy links.
- If one frame is damaged, Go-Back-N may resend many correct frames.
- Selective Repeat resends only the damaged or missing frame.
- This improves efficiency on noisy links.
- The cost is more complex receiver processing and buffering.

Core difference

Go-Back-N discards out-of-order frames; Selective Repeat can store them until the missing gap is filled.

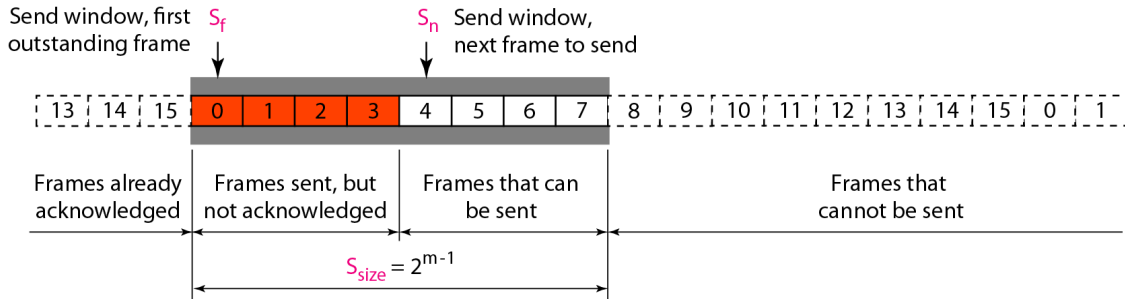
Selective Repeat ARQ: Windows and NAKs

- Selective Repeat uses both a send window and a receive window.
- The send and receive windows have the same size.
- With m -bit sequence numbers, each window is at most 2^{m-1} .
- Out-of-order frames inside the receive window may be stored.
- A NAK asks the sender to retransmit a specific missing or damaged frame.

The receiver is now an active buffer, not just an in-order filter.

Selective Repeat Send Window

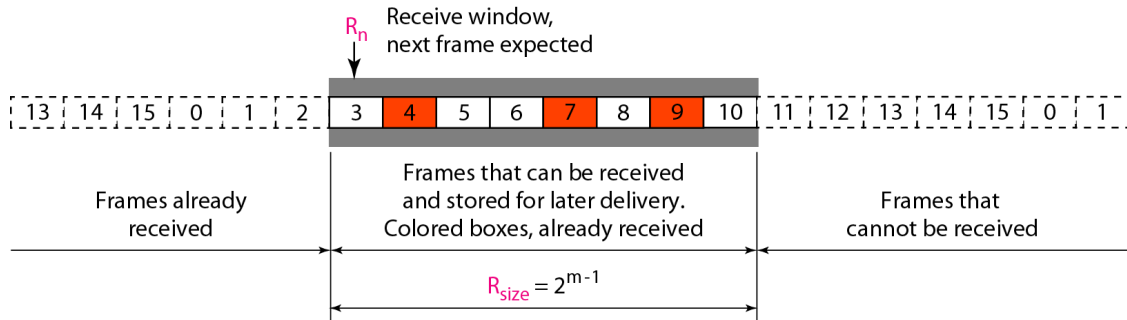
Figure: 10.18 Send window for Selective Repeat ARQ



The sender tracks multiple outstanding frames, but each frame may be acknowledged or retransmitted independently.

Selective Repeat Receive Window

Figure: 10.19 Receive window for Selective Repeat ARQ



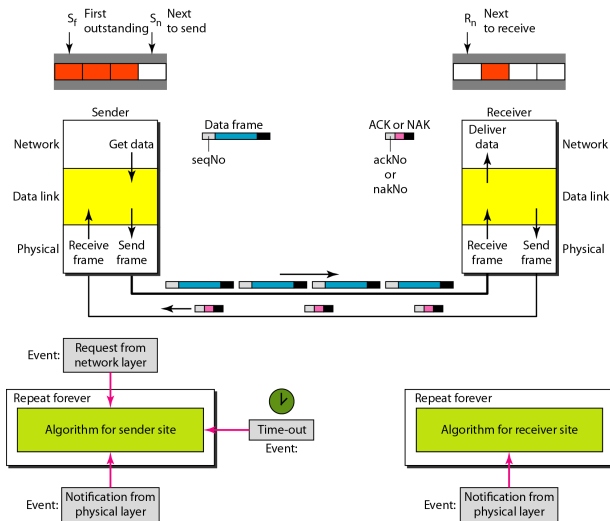
The receiver can accept and buffer several sequence numbers, including out-of-order frames.

Selective Repeat ARQ Design

Figure: 10.20 Design of Selective Repeat ARQ

- Every outstanding frame carries its own timer.
- The receiver keeps a window of buffered frames, not a single slot.
- NAKs name one specific frame to resend.

Selective Repeat requires per-frame state, per-frame timers, and receiver buffering.

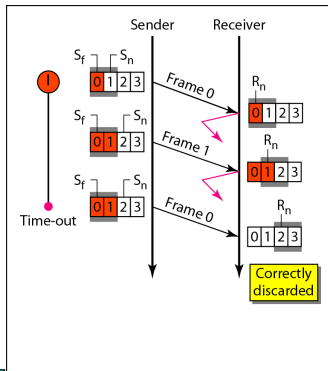


Selective Repeat Window Size

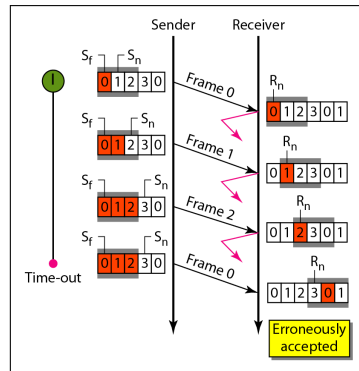
Figure: 10.21 Selective Repeat ARQ window size

$$\text{Send} = \text{receive} \leq \frac{2^m}{2} = 2^{m-1}$$

In (a) the window is 2^{m-1} and the duplicate is correctly discarded. In (b) a larger window lets the receiver accept the same duplicate as new data.



a. Window size = 2^{m-1}



b. Window size > 2^{m-1}

In Selective Repeat ARQ, the sender and receiver window sizes must be at most 2^{m-1} , which is half of the total 2^m sequence numbers. If the window is larger than this, sequence numbers can wrap around too soon. Then, if ACKs are lost and the sender retransmits an old frame, that old sequence number may already fall inside the receiver's current window, causing the receiver to mistake the duplicate frame for new data. Limiting the window to 2^{m-1} prevents this ambiguity.

Algorithm 10.9: Selective Repeat Sender

```
Sf <- 0                      // first outstanding frame
Sn <- 0                      // next frame number to use
Ssize <- 2(m-1)

while true do
  wait for an event

  if event == request_from_network_layer and Sn is in send_window then
    packet <- get_data()
    frame <- make_frame(packet, seq = Sn)
    store_copy(frame)
    send_frame(frame)
    start_timer(Sn)
    Sn <- (Sn + 1) mod 2m
  end if

  if event == timeout(frame_number) then
    resend_stored_frame(frame_number)
    restart_timer(frame_number)
  end if
// the same event loop continues on the next slide
```

Each outstanding frame is stored separately and gets its own timer, so a timeout names exactly one frame to resend.

Algorithm 10.9: Selective Repeat Sender (ACK/NAK)

```
// continued: the same event loop started on the previous slide

if event == ack_arrival and ack_is_valid then
    mark_frame_as_acknowledged(ack_number)
    stop_timer(ack_number)

    while frame Sf is acknowledged do
        discard_copy(Sf)
        Sf <- (Sf + 1) mod 2m
        slide_send_window()
    end while
end if

if event == nak_arrival and nak_is_valid then
    n <- nak_number
    resend_stored_frame(n)
    restart_timer(n)
end if
end while
```

Selective Repeat retransmits a named frame instead of returning to the first outstanding frame and resending everything.

Algorithm 10.10: Selective Repeat Receiver (Arrival)

```
Rn <- 0                                // first frame expected for delivery
Rsize <- 2^(m-1)
nak_sent <- false

while true do
  wait for frame arrival
  frame <- receive_frame()

  if frame_is_valid(frame) and seq(frame) is in receive_window then
    store_and_mark(frame)

    if seq(frame) != Rn and not nak_sent then
      send_nak(Rn)
      nak_sent <- true
    end if

    // delivery and acknowledgment continue on the next slide
```

A valid frame inside the receive window is stored. If it reveals a gap, the receiver sends one NAK for the missing first frame.

Algorithm 10.10: Selective Repeat Receiver (Delivery)

```
// continued: still inside "if frame_is_valid and seq in receive_window"

    while frame Rn is marked do
        deliver_data(extract_data(frame Rn))
        unmark(Rn)
        Rn <- (Rn + 1) mod 2m
        slide_receive_window()
        nak_sent <- false
    end while

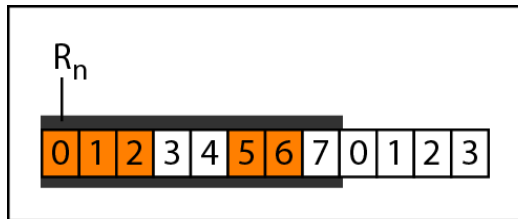
    send_ack(Rn)

else if not nak_sent then          // corrupted or outside the window
    send_nak(Rn)
    nak_sent <- true
end if
end while
```

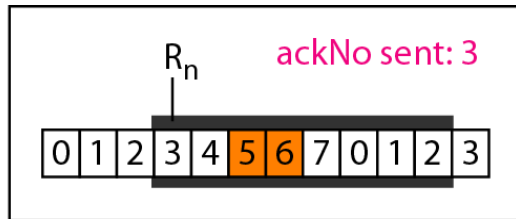
Only consecutive stored frames starting at R_n are delivered upward; then the receive window slides.

Selective Repeat Delivery Rule

Figure: 10.22 Delivery of data in Selective Repeat ARQ



a. Before delivery



b. After delivery

Frames may be accepted out of order, but delivery upward occurs only for a consecutive run starting at the beginning of the receive window.

Example 10.8: Lost Frame in Selective Repeat

- The example is similar to the earlier lost-frame case.
- Frame 1 is lost.
- Each sent or resent frame has its own timer.
- Timer 0 stops when ACK 0 arrives.
- Timer 1 restarts when a NAK arrives and stops when the final ACK arrives.
- Timers for frames 2 and 3 stop at the last arrival event.

Example 10.8: Receiver Buffering

- Acceptance and delivery are different actions.
- Frame 2 arrives and is stored, but it cannot be delivered because frame 1 is missing.
- Frame 3 is also stored, but delivery still waits.
- When frame 1 finally arrives, frames 1, 2, and 3 can be delivered together.
- Delivery requires a consecutive set of frames starting at the beginning of the receive window.

Example 10.8: NAK and ACK Behavior

- A NAK is sent after the receiver first detects that frame 1 is missing.
- A second similar situation does not trigger another NAK.
- The protocol remembers that NAK 1 has already been sent for this window position.
- ACKs are sent when data are delivered to the network layer.
- If several frames are delivered together, one ACK can acknowledge all of them.

Selective Repeat ARQ allows the sender to transmit multiple frames without waiting for an acknowledgement after every frame. Both the sender and receiver use sliding windows, and the receiver can accept and temporarily store out-of-order frames as long as their sequence numbers are inside the receive window. Each transmitted frame is tracked separately and normally has its own timer.

When a frame is lost, later frames do not have to be discarded. For example, if Frames 0, 1, 2, and 3 are sent and Frame 1 is lost, the receiver may still receive Frames 2 and 3 correctly. It stores Frames 2 and 3 in its buffer but cannot deliver them yet because Frame 1 is missing. The receiver may send NAK 1 to request retransmission of Frame 1. The sender then retransmits only Frame 1 rather than retransmitting Frames 1, 2, and 3 as Go-Back-N would.

When the retransmitted Frame 1 arrives, the receiver now has Frames 1, 2, and 3 consecutively. It can therefore deliver all of them in the correct order and move its receive window forward. If it now expects Frame 4, it sends ACK 4, indicating that all frames up to Frame 3 have been successfully received and that Frame 4 is the next expected frame.

If several frames are lost, they are handled independently. For example, if Frames 1 and 2 are lost but Frame 3 arrives, Frame 3 is stored while the receiver waits for Frames 1 and 2. Frame 1 and Frame 2 are retransmitted separately through NAKs or when their individual timers expire. Once the missing frames arrive, the receiver combines them with the already buffered Frame 3 and delivers the complete sequence in order.

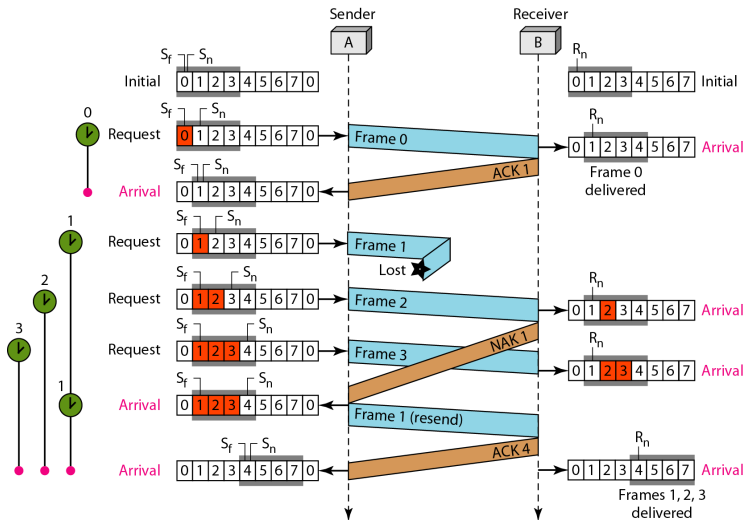
The main advantage of Selective Repeat ARQ is that only the specific missing or damaged frames are retransmitted. Correctly received out-of-order frames are saved rather than discarded, which avoids unnecessary retransmissions and makes Selective Repeat more bandwidth-efficient than Go-Back-N. However, it is more complex because the sender must maintain separate information and timers for multiple frames, while the receiver needs buffer space to store out-of-order frames.

Example 10.8 Flow Diagram

Figure: 10.23 Flow diagram for Example 10.8

- Frame 1 is lost; frames 2 and 3 are stored, not discarded.
- One NAK 1 asks for the missing frame only.
- Frames 1, 2, and 3 are delivered together once frame 1 arrives.

Out-of-order frames are buffered until the missing frame arrives; then delivery catches up.



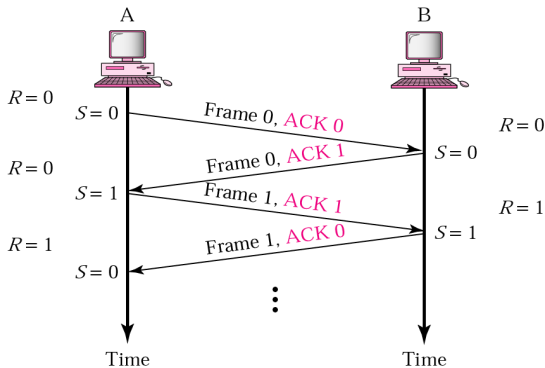
Piggybacking

Definition

Piggybacking combines outgoing data and acknowledgment information in one frame.

- Useful when communication is bidirectional.
- A station acknowledges received frames while sending its own data.
- Separate ACK traffic is reduced.
- Go-Back-N ARQ and HDLC both use this idea.

Figure: Data and ACK traveling together



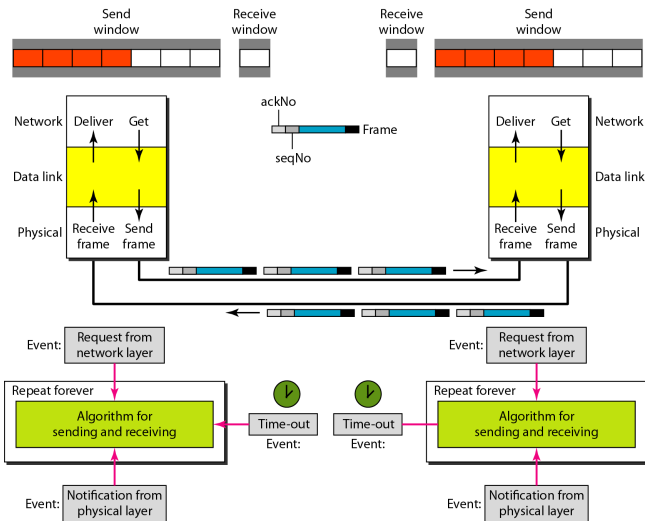
The acknowledgment is carried together with data traveling in the opposite direction.

Piggybacking in Go-Back-N ARQ

Figure: 10.24 Design of piggybacking in Go-Back-N ARQ

- Each station now owns both a send window and a receive window.
- One frame carries this station's data and the other station's acknowledgment.
- The design is symmetric: both ends run the same algorithm.

Data and acknowledgment fields share one bidirectional frame exchange.



High-level Data Link Control

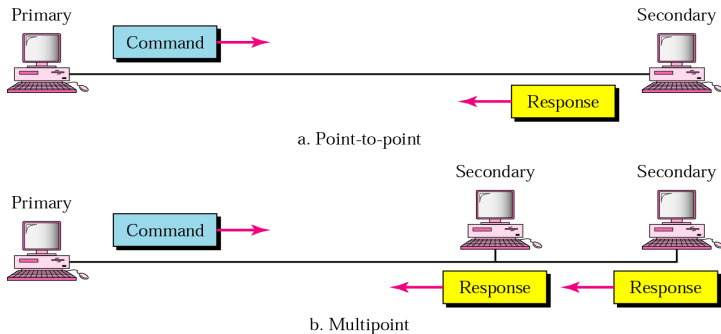
HDLC is a bit-oriented data-link protocol. A mode defines the relationship between stations and who controls the link.

- A mode describes the control relationship between devices.
- HDLC supports Normal Response Mode (NRM).
- HDLC supports Asynchronous Balanced Mode (ABM).
- These modes determine whether a station needs permission before transmitting.

HDLC Normal Response Mode

- NRM uses a primary-secondary relationship.
- A secondary station transmits only after permission from the primary station.
- The primary station controls the exchange.

Figure: 10.25 Normal response mode, point-to-point and multipoint

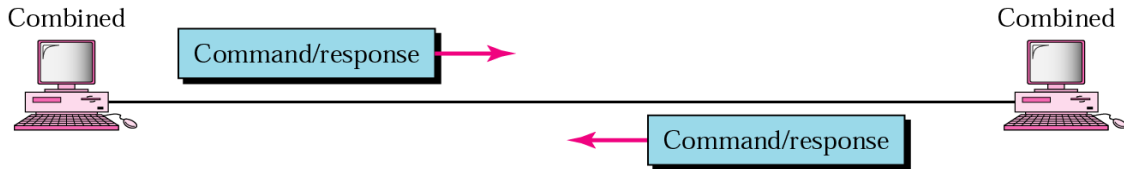


NRM is centralized link control.

HDLC Asynchronous Balanced Mode

- ABM treats stations as equals.
- Combined stations are used on a point-to-point link.
- Either station may initiate transmission without waiting for permission.

Figure: 10.26 Asynchronous balanced mode

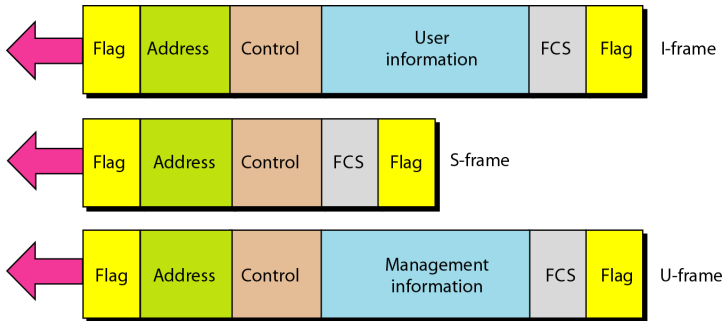


ABM is balanced peer-to-peer link control: each station is a combined station that both commands and responds.

HDLC Frame

- HDLC frames begin and end with the same flag pattern.
- Fields carry addressing, control, user data, and error detection.
- The control field determines the frame type and control meaning.

Figure: 10.27 HDLC frames: I-frame, S-frame, and U-frame



HDLC Frame Types

I-frame

Carries user data and control information related to that data.

S-frame

Carries supervisory control information, especially flow and error control.

U-frame

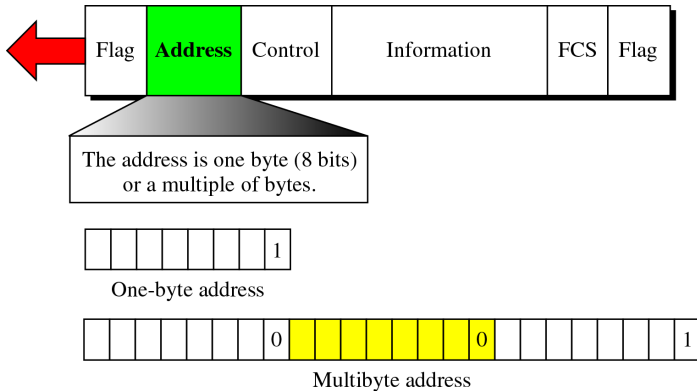
Carries unnumbered management information for the link itself.

HDLC separates data transfer, supervision, and link management into different frame roles.

HDLC Fields: Flag and Address

- A frame may contain up to six fields.
- The flag field is the bit pattern 01111110.
- The flag identifies both frame start and frame end.
- The address field identifies the secondary station that is the source or destination.

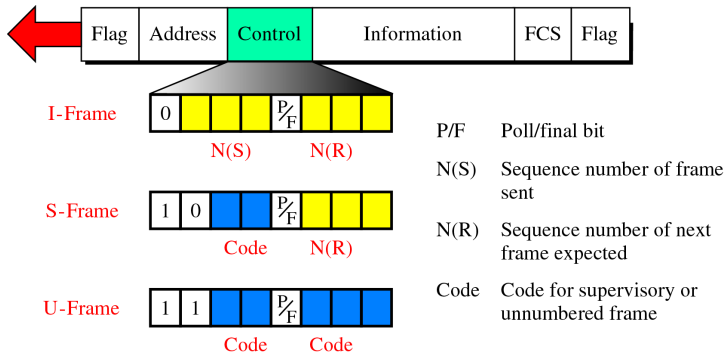
Figure: 10.28a Flag and address fields



HDLC Control Field

- The control field is 1 or 2 bytes.
- It supports flow control and error control.
- Interpretation depends on frame type.
- I, S, and U frames use different bit layouts.

Figure: 10.28b Control field format for the three frame types



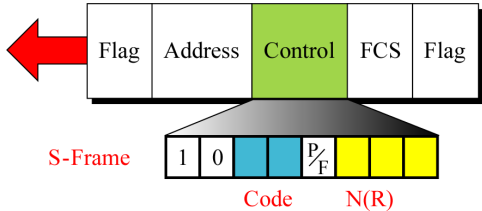
Polling is useful when one station needs to control access to the link or explicitly ask another station to respond. In HDLC Normal Response Mode, the primary station may have several secondary stations connected to it. If every secondary transmitted whenever it wanted, their transmissions could interfere with each other. So the primary polls one secondary at a time, essentially saying, "You may respond now." That secondary can then send data or status information.

Without polling in NRM, the secondary stations would not have a clear indication of when they are allowed to transmit, so link control could become ambiguous or multiple secondaries could try to transmit at the same time. Polling therefore provides orderly, centralized control of communication.

In ABM, polling is less essential because both stations are equal and can initiate transmission independently. There, the P/F bit is mainly useful when one station specifically wants an immediate response or status from the other.

HDLC S-Frame and U-Frame Control

Figure: 10.28c S-frame control field

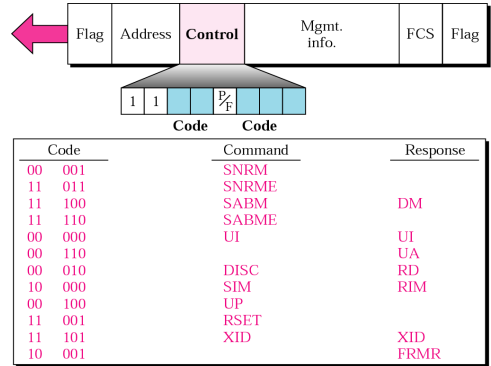


examples:
REJ, N(5): resend
from frame 5
SREJ, N(5): resend
just frame 5

Code	Command
00	RR Receive ready
01	REJ Reject
10	RNR Receive not ready
11	SREJ Selective-reject

S-frames carry acknowledgments, flow control, and error control.

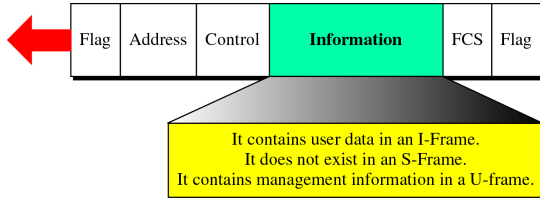
Figure: 10.28d U-frame control field



U-frames carry session management and link-control information.

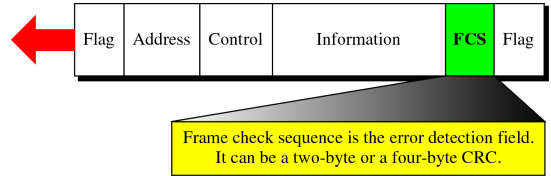
HDLC Information and FCS Fields

Figure: 10.28e Information field



The information field carries network-layer data or management information.

Figure: 10.28f Frame check sequence field



The FCS is the HDLC error-detection field: a two-byte or four-byte CRC.

Table 10.1: U-Frame Commands and Responses

Command	Purpose	Response	Purpose
SNRM	Set Normal Response Mode	UA	Unnumbered acknowledgment
SARM	Set Asynchronous Response Mode	DM	Disconnected mode
SABM	Set Asynchronous Balanced Mode	RIM	Request initialization mode
DISC	Disconnect the link	RD	Request disconnect
SIM	Set initialization mode	FRMR	Frame reject
UP	Unnumbered poll	UI	Unnumbered information

U-frames manage link setup, release, initialization, and exceptional control conditions rather than carrying ordinary numbered data.

Example 10.9: HDLC Connection and Release

- Node A requests a connection using SABM.
- Node B accepts with UA.
- Data transfer can occur after this setup exchange.
- Node A later releases the connection with DISC.
- Node B confirms the release with UA.

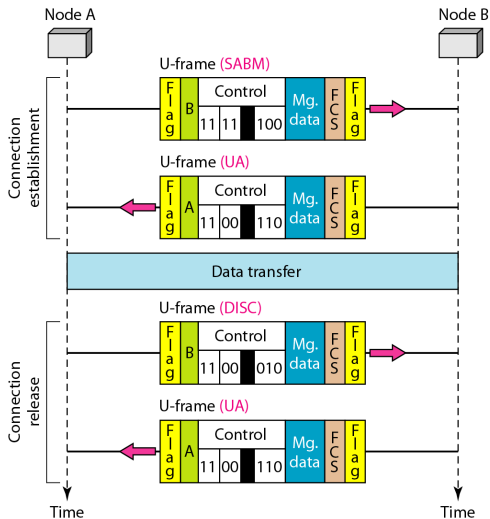
U-frames handle link management before and after data transfer.

Example 10.9 Diagram

Figure: 10.29 Example of connection and disconnection

- Node A opens the link with a U-frame carrying SABM.
- Node B accepts with UA, and data transfer begins.
- Node A closes the link with DISC; B confirms with UA.
- All four control frames are U-frames, so they carry no sequence numbers.

SABM/UA establishes the link; DISC/UA releases it.



Example 10.10: Piggybacking without Error

- Node A sends I-frame 0 and then I-frame 1.
- Node B piggybacks acknowledgment of both A frames onto its own I-frame.
- B's first I-frame has $N(S) = 0$ and $N(R) = 2$.
- $N(R) = 2$ means B has accepted A's frames 0 and 1 and expects A's frame 2 next.
- B sends its I-frames numbered 1 and 2 before accepting more frames from A.

Example 10.10: Supervisory ACK

- B's $N(R)$ value remains 2 while it is still expecting A's frame 2.
- Node A has no more data to send.
- Because A cannot piggyback the acknowledgment onto an I-frame, it sends an S-frame.
- The RR code says A is ready to receive.
- $N(R) = 3$ tells B that frames 0, 1, and 2 have been accepted and frame 3 is expected next.

First, A sends I-frame 0 and I-frame 1 to B. Since these are I-frames, each carries A's user data and A's own sequence number $N(S)$. After B receives both correctly, B now expects A's Frame 2, so B's acknowledgement value becomes: $N(R) = 2$

Instead of sending a separate ACK, B has its own data to send back to A. So B piggybacks that acknowledgement inside its own I-frame:

B $\rightarrow$ A : I-frame with $N(S)=0$, $N(R)=2$

This means:

> "This is my Frame 0, and I have received your Frames 0 and 1; I expect your Frame 2 next."

B then continues sending its own I-frames 1 and 2. While doing so, ' $N(R)$ ' can remain 2 because B is still waiting for A's Frame 2. So the ACK information and B's data travel together in the same I-frames.

Now A receives B's Frames 0, 1, and 2 correctly. Therefore A now expects B's Frame 3, so A needs to acknowledge with:

$N(R) = 3$

But A has no more data of its own to send**, so it cannot piggyback this ACK onto an I-frame. Instead, A sends a separate S-frame, specifically: RR, $N(R)=3$

RR means Receive Ready, and $N(R)=3$ means:

> "I have successfully received your Frames 0, 1, and 2, and I am ready for Frame 3."

So the key idea of the diagram is:

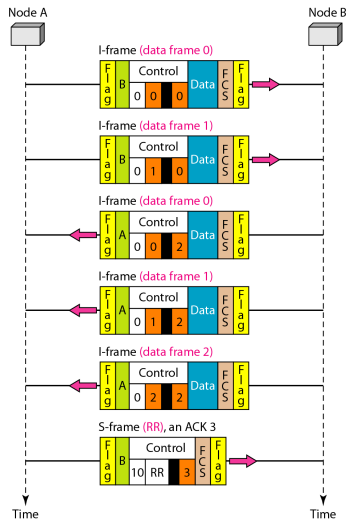
If a station has data to send, it puts the ACK inside its outgoing I-frame. If it has no data to send, it sends a separate S-frame such as RR to carry the acknowledgement.

Example 10.10 Diagram

Figure: 10.30 Example of piggybacking without error

- A sends I-frames 0 and 1; B acknowledges both inside its own I-frame.
- B's first I-frame carries $N(S) = 0$ and $N(R) = 2$, meaning "I have taken your 0 and 1, send me 2".
- A has no more data, so it acknowledges with an S-frame instead: RR with $N(R) = 3$.

When data are available in both directions, ACK information can ride inside outgoing I-frames.



Example 10.11: Piggybacking with Error

- Node B sends data frames 0, 1, and 2.
- Frame 1 is lost.
- Node A receives frame 2 out of order and discards it.
- A sends a REJ frame for frame 1.
- REJ both confirms frame 0 and asks B to resend frame 1 and following frames.
- B retransmits frames 1 and 2; A later acknowledges with RR and acknowledgment number 3.

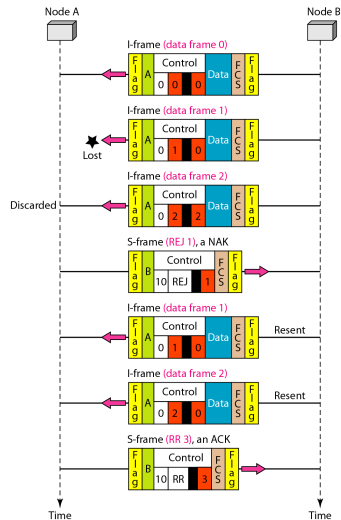
This example uses Go-Back-N behavior with REJ acting as a negative acknowledgment.

Example 10.11 Diagram

Figure: 10.31 Example of piggybacking with error

- B sends data frames 0, 1, and 2; frame 1 is lost.
- A receives frame 2 out of order and discards it.
- A returns REJ with $N(R) = 1$: frame 0 is confirmed, frame 1 onward must be resent.
- B retransmits frames 1 and 2, and A finally acknowledges with RR and $N(R) = 3$.

A rejected frame causes retransmission from the missing sequence number onward, which is Go-Back-N behavior.



Summary (1)

- Data link control defines procedures for communication between two adjacent nodes.
- Frames can be fixed size or variable size.
- Fixed-size framing does not need explicit frame-boundary delimiters.
- Variable-size framing needs a delimiter, often called a flag.
- Character-oriented protocols view the data field as bytes and use byte stuffing.
- Bit-oriented protocols view the data field as bits and use bit stuffing.
- Flow control limits how much a sender may transmit before feedback.
- Error control combines detection with correction, commonly by retransmission.

Summary (2)

- For noiseless channels, the chapter introduced the Simplest Protocol and Stop-and-Wait Protocol.
- For noisy channels, the chapter introduced Stop-and-Wait ARQ, Go-Back-N ARQ, and Selective Repeat ARQ.
- Go-Back-N and Selective Repeat both use sliding windows.
- Piggybacking improves bidirectional efficiency by combining data and acknowledgment information.
- HDLC is a bit-oriented protocol for point-to-point and multipoint links.
- The Point-to-Point Protocol (PPP) is a common byte-oriented protocol for point-to-point access.

Review prompts

- Why is stuffing necessary?
- How do flow control and error control differ?
- Why is Stop-and-Wait inefficient on long-delay links?
- What does Go-Back-N retransmit after a loss?

Check your intuition

- Why does Selective Repeat need a receive window?
- Why must Selective Repeat use only half the sequence space?
- When is piggybacking useful?
- What roles do HDLC I, S, and U frames play?

